

# Module 2 Reliable Python Software

## Overview and study route

Software Engineering in the Era of AI | Master's year one | Electrical and Information Engineering | Weeks 3 and 4 | 2026 to 2027

A calculation becomes engineering software when another person can understand its contract, run it with new data, diagnose a failure and change it without silently breaking existing behaviour. This module develops those capabilities through modular design, explicit error handling, version control and independent testing.

In Week 3, separate the energy calculation from input and reporting. In Week 4, construct evidence that challenges the implementation. The analytical emphasis is the test oracle: the rule or reference used to decide whether an output is correct. A passing test is only useful when its expectation is justified independently of the code it checks.

## Learning outcomes

- Decompose a small application into cohesive functions and modules with explicit interfaces.
- Explain mutability, side effects, exceptions and file boundaries in supplied Python.
- Use a small Git change and review its diff before integration.
- Derive normal, boundary, invalid and numerical test cases from requirements.
- Distinguish unit, integration, system and acceptance evidence.
- Explain the limits of coverage and apply regression, property or mutation reasoning.
- Prepare a project proposal with a runnable deterministic baseline.

These outcomes support LO1, LO2, LO3 and LO7. The prerequisite is the function and list work in Module 1.

## 1 Design for a change you can explain

A cohesive function performs one related responsibility. Coupling describes dependencies between components. A sensor loader must depend on a file format, while an energy calculation need not. If the calculation reads a particular file, prints a report and calls an AI service, a small mathematical change becomes entangled with storage, display and network behaviour.

Use a thin input boundary, a deterministic computation and a separate presentation layer. The input boundary interprets CSV fields and rejects malformed records. The core function calculates energy under its documented contract. The report layer chooses wording and display units. Each layer has a reason to change that can be explained separately.

| Component               | Responsibility                                  | Failure example                       |
|-------------------------|-------------------------------------------------|---------------------------------------|
| CSV loader              | Read and validate the external record structure | Missing power_W column                |
| Energy function         | Compute Wh from valid interval-average readings | Invalid interval or non-finite result |
| Report function         | Format an already computed result               | Wrong displayed unit                  |
| Application entry point | Coordinate the components                       | Missing input path                    |

A modular design is not a demand for many files. A ten-line computation may need only one small module. Abstraction becomes useful when it isolates a real change or test boundary. Avoid creating a hierarchy of classes merely because an agent can generate one quickly.

## 2 Modules imports and execution

A Python module is commonly a .py file containing definitions. Importing it gives another module access to those definitions. Code at the top level can run during import, so avoid unnecessary file writes, network calls or display output there. The conventional main guard keeps command-line behaviour separate from reusable definitions.

```
from energy import interval_energy_Wh

def main():
    result = interval_energy_Wh([60.0, 120.0, 0.0], 30.0)
    print(f"Energy: {result:.2f} Wh")

if __name__ == "__main__":
    main()
```

This application prints Energy: 90.00 Wh. The f-string inserts the expression between braces and formats it with two decimal places. Formatting changes the representation, not the stored result. Two decimals may be a convenient display choice, but measurement resolution should determine how many digits are meaningful.

Use descriptive module names. Naming a local file math.py or csv.py can hide a standard-library module with the same name. When an import behaves unexpectedly, inspect the executed environment and imported file location before assuming the library is broken.

## 3 Data ownership and side effects

An operation has a side effect when it changes something beyond its returned value: writing a file, mutating an input list, printing, or updating a global variable. Side effects are not inherently wrong; applications need them. They are easier to reason about when concentrated at explicit boundaries.

Suppose an energy function sorts the caller's readings in place. The total energy may remain unchanged for equal intervals, yet timestamp alignment could be destroyed for later operations. A function should not mutate its inputs unless its contract explicitly says so. A test should verify both the result and relevant unchanged state.

Lists are mutable. Assigning `b = a` makes `b` refer to the same list, not an independent copy. For a flat list, `b = list(a)` creates a separate outer list. Nested structures need more careful ownership reasoning because

an outer copy still shares nested objects. In this course, prefer immutable records or small clearly owned collections over clever copying.

Pure functions, which produce results from inputs without observable side effects, are easier to test. Determinism additionally requires that hidden sources such as time or random state do not influence the output. Passing time, configuration or a provider into a function makes those dependencies visible and replaceable in tests.

## 4 Validate boundaries and preserve failure meaning

External data is text or bytes before it becomes trusted application values. A CSV record might contain an empty field, a string that looks numeric, an unexpected unit, or a duplicate timestamp. Validation should check structure, conversion and domain separately. Successful float conversion does not guarantee a finite or physically permitted reading.

Reject a malformed record with a useful reason and row identifier. Do not include secrets or unnecessary personal data in the diagnostic. Decide explicitly whether one invalid row rejects the entire batch or produces a partial report. Our core energy service rejects the batch because reporting a complete total from incomplete input would mislead its user.

Use `ValueError` for values outside a function's contract. Catch specific expected exceptions at the application boundary when you can take a defined action, such as reporting invalid input and returning a non-success status. Catching every `Exception` and continuing can conceal programming defects.

The input boundary should not silently infer units from magnitude. A value of 1000 could be watts or milliwatts; magnitude is not reliable metadata. Store or specify the unit and convert through an explicit rule.

## 5 Git as an inspectable change history

Git records snapshots and relationships among versions. A working tree contains current files; the staging area selects content for the next commit; a commit records a snapshot and message. A branch identifies a line of work. A diff shows what changed between states.

For a local exercise, initialise a repository, add only the intended files, commit a baseline, make one bounded change, inspect the diff and commit the change with a reason. Git's official book explains these concepts and commands. [R1] Avoid committing access keys, local environments, generated caches or private datasets.

```
git init
git add energy.py test_energy.py
git commit -m "Add interval energy calculation and contract tests"
git diff
git status
```

These commands are a teaching sequence; configure the approved identity before the first commit. The exact files in a project may differ. Read the output of status rather than assuming that everything in the folder belongs in a commit.

A reviewable change has a focused requirement, a small diff and evidence. The reviewer asks whether the behaviour matches the contract, whether failure handling is meaningful and whether tests challenge the change. An AI-generated review comment is a hypothesis to inspect, not an automatic finding.

When two edits conflict, decide the intended combined behaviour and rerun relevant tests after resolving the text. A conflict-free merge can still be semantically wrong. For example, one branch may change units while another adds a report that assumes the old units.

## 6 The test oracle and independence

A test consists of a controlled setup, an action and an assertion about the result or resulting state. Its oracle may be an analytic solution, a trusted reference implementation, a physical invariant, a human-reviewed label or an external specification. Choose the oracle before examining a candidate output when possible.

For energy, manually calculate [100, 200] W over two 15-minute intervals as 75 Wh. If the implementation instead computes 150 and the test copies that number, both artifacts agree while both violate the requirement. Having one model generate implementation and tests can amplify this correlated error.

Organise cases by equivalence classes. Valid positive readings form one useful class; zero power probes an allowed boundary; empty input and negative duration probe invalid classes. Add cases suggested by the representation, such as NaN, infinity, booleans and strings. A class partition is a reasoning aid, not proof that every member behaves identically.

## 7 Levels of testing

A unit test checks a small component in isolation. An integration test checks the interaction between components, such as CSV parsing followed by calculation. A system test exercises an assembled application. An acceptance test checks behaviour that matters to a stakeholder under specified conditions.

The distinction is about the boundary and purpose, not the testing framework. One command-line test may be both a system test and evidence for an acceptance criterion. Keep fast deterministic tests frequent and use slower environment-dependent checks where they address a distinct risk.

For an AI adapter later in the course, a mock or stub can test that timeouts produce the required fallback. It cannot establish that a live model produces useful explanations. Code contract tests and live-output evaluations answer different questions.

## 8 Write meaningful assertions

The supplied `test_energy.py` uses the standard-library `unittest` runner so that the core exercise has no installation dependency. The same cases can be written in `pytest`, whose assertion and exception documentation is a useful reference. [R2]

```
import math
from energy import interval_energy_Wh

result = interval_energy_Wh([100.0, 200.0], 15.0)
assert math.isclose(result, 75.0, rel_tol=1e-12, abs_tol=1e-9)
```

An absolute tolerance limits the allowed difference in the result's unit. A relative tolerance limits the difference in proportion to the result's magnitude. Near zero, an absolute tolerance is often necessary. The illustrative tolerances above address numerical arithmetic in a small exact classroom case; they are not a specification of sensor accuracy.

Python assert statements may be disabled in optimised execution. Therefore, use explicit checks that raise exceptions for production input validation. Assertions are appropriate in the shown test context; they should not be the only protection on an external input boundary.

An invalid-input test must assert that the intended error occurs. Merely calling the function without checking the result is not a test. Also verify that a failed operation did not leave a partial output file or partially updated state where that matters.

## 9 Regression properties and mutation

A regression test preserves evidence for a behaviour that previously failed or must remain stable. After fixing minute-to-hour conversion, keep a non-hourly interval case so a later simplification does not reintroduce the defect.

A property states a relationship over many valid inputs. For equal-duration intervals, doubling all powers doubles energy within numerical tolerance. Concatenating two valid sequences gives the sum of their energies, provided the interval and interpretation agree. Reordering readings preserves total energy in this equal-interval model but does not preserve a time-series plot or peak timing.

Property-based testing tools can generate many inputs from a stated domain. For the core exercise, manually test several instances of a property. Do not use a property that also holds for a faulty program as the sole oracle: a function that always returns zero satisfies some scaling relations.

Mutation reasoning asks whether tests detect a deliberate small defect. Consider removing division by 60, changing a rejection condition from less than zero to less than or equal to zero, or returning mean power. A good suite should detect each relevant mutation. If a mutation survives, inspect whether the suite lacks a case or the mutation is behaviourally equivalent within the domain.

Coverage records what code ran during tests. Full line coverage does not establish correct assertions, adequate input classes or valid requirements. Treat coverage as a way to find unexamined code, not a certificate of correctness.

## 10 Debugging as an experiment

State a hypothesis, choose a discriminating input and predict what each possible cause would produce. If an energy report is too large by sixty, compare a one-minute and a sixty-minute case. If both are wrong in the same ratio, investigate unit conversion. If only a mixed input fails, investigate parsing or state.

Reduce the failure while preserving the behaviour. A three-row CSV is easier to inspect than a thousand-row file. Keep the failing example, change one cause and rerun relevant checks. Record the root cause in terms of the contract, not only the line changed.

At master's level, evaluate the strength of the evidence: does the fix address a whole class of failures or just a special case? A patch that recognises one file name and returns the expected result passes a weak test but does not implement the required computation.

## Weekly laboratories

### Week 3 lab Separate and review a change

Preparation: run the Module 1 reference function. Use Guided assistance for unfamiliar syntax and Bounded development only for the small change assigned by the instructor.

- Minutes 0 to 10: identify computation, input and presentation responsibilities.
- Minutes 10 to 25: write a separate formatting function that returns a labelled Wh string.
- Minutes 25 to 40: create a local baseline and inspect the diff for the new function.
- Minutes 40 to 50: exchange reviews; each reviewer checks that the calculation and units remain unchanged.
- Minutes 50 to 60: commit the reviewed change and record one accepted or rejected review suggestion.

Submit the source, diff or commit identifier, and a short review note. An extension is to load a supplied CSV and explicitly reject one malformed row.

## Week 4 lab Test a candidate and submit the proposal

Preparation: derive at least three expected results without running the candidate.

- Minutes 0 to 10: compare the stated contract with the candidate function.
- Minutes 10 to 30: run the supplied tests and add one missing boundary or invalid case.
- Minutes 30 to 45: introduce a deliberate unit-conversion mutation in a disposable copy and show which test detects it.
- Minutes 45 to 55: explain why passing tests do not prove a valid physical model.
- Minutes 55 to 60: submit the project proposal and initial executable baseline.

The proposal identifies the engineering need, data, deterministic baseline, possible AI contribution and evaluation question. It is the first project checkpoint, not an additional assessment weight.

## Practice questions

### Question 1 Decomposition

A function reads CSV, calls an AI model, computes energy, updates a database and prints a report. Propose three boundaries and justify them.

**Model answer.** Separate input parsing and validation, deterministic domain computation, and external effects such as persistence or AI advice. Presentation can be another boundary. This permits the calculation to be tested without network or database access and makes each failure policy explicit. Equivalent coherent decompositions are acceptable.

### Question 2 Mutation of input

A calculation returns the correct total but sorts the input list in place. Why might this still be a defect?

**Model answer.** Sorting can destroy association with timestamps or other parallel records and violates a non-mutating contract. Test both the returned result and preservation of the original collection. Correctness includes relevant state, not just one number.

### Question 3 Independent oracle

For [30, 90] W at 20-minute intervals, derive the expected energy before writing a test.

**Model answer.** Each interval is one third of an hour. Energy is  $(30 + 90) / 3 = 40$  Wh. This hand calculation supplies an oracle independent of the implementation.

## Question 4 Boundary selection

Choose four tests that distinguish a valid zero result from invalid data.

**Model answer.** [0] with a positive interval returns 0 Wh. Empty input is rejected. A negative reading is rejected. A zero interval is rejected. Include the expected outcome for each; testing only positive values misses the distinction.

## Question 5 Numerical tolerance

Why can relative tolerance alone be unsuitable when the expected result is zero?

**Model answer.** The permitted difference scales with magnitude and may become zero or unreasonably small near zero. An absolute tolerance sets a physically or numerically justified bound in the result's unit. Its value must fit the context.

## Question 6 Coverage

Every line executes during a test, but the test never checks the output. What has been established?

**Model answer.** Only that the exercised path completed under that setup without an uncaught failure. It has not established the required result. Add justified assertions and cases that challenge the contract.

## Question 7 Correlated errors

An assistant writes a wrong formula and generates tests using that same formula for expected results. What is the problem and how can you reduce it?

**Model answer.** The tests share the implementation's assumption, so agreement is weak evidence. Derive independent examples, review the specification, use a separate trusted oracle or physical relationship, and keep some acceptance cases hidden from the implementation task.

## Question 8 Git review

An AI change includes a useful bug fix, a new dependency and a modified evaluation answer file. What should a reviewer investigate?

**Model answer.** Check whether the dependency is necessary and authentic, why the answer file changed, and whether the fix passes untouched independent tests. Separate unrelated edits and reject changes that invalidate the evaluator. A passing result after rewriting expected answers is not evidence of repair.

## Question 9 Test levels

Classify a direct call to the energy function, CSV-to-energy processing and an operator's acceptance scenario.

**Model answer.** The direct call is a unit test. CSV parsing followed by computation is an integration test. The operator scenario is acceptance evidence and may run at system level. Categories can overlap in purpose; explain the boundary.

## Question 10 Surviving mutation

All tests use a 60-minute interval. Why might replacing the energy formula with `sum(powers_W)` survive?

**Model answer.** For one-hour intervals the duration factor is one, so the faulty simplification produces the same number. Add a non-hourly case such as [100, 200] at 15 minutes, whose result is 75 Wh rather than 300 Wh.

## Question 11 Property limits

Does passing the doubling property prove that an energy function is correct?

**Model answer.** No. An always-zero function or a wrongly scaled sum may also satisfy it. Combine properties with anchored numerical examples, domain checks and review of the model.

## Question 12 Failure handling

A CSV loader catches all errors and returns an empty list; the application reports zero energy. Evaluate this policy.

**Model answer.** It converts a failed or missing dataset into apparently valid zero consumption. Preserve a distinct failure result or reject the batch with a meaningful reason. A partial-data policy would need explicit completeness information and acceptance criteria.

## Revision checklist

Explain a module boundary and a side effect. Derive a test oracle manually. Distinguish boundary testing from coverage. Review a diff for unrelated changes. State a mutation your tests detect. Explain one failure that mocks cannot reveal. Present a small executable baseline for the project proposal.

## Sources and further reading

- [R1] Chacon and Straub. Pro Git, official online book. <https://git-scm.com/book/en/v2>
- [R2] pytest. Assertions and expected exceptions. <https://docs.pytest.org/en/stable/how-to/assert.html>
- Python Software Foundation. Modules, exceptions and unittest documentation. <https://docs.python.org/3/tutorial/modules.html> and <https://docs.python.org/3/library/unittest.html>
- Further reading: Winters, Manshreck and Wright, Software Engineering at Google, selected chapters on testing and code review.

References checked 15 September 2026. Examples are classroom constructions. The local test suite checks the supplied implementation; it is not a claim of production certification.